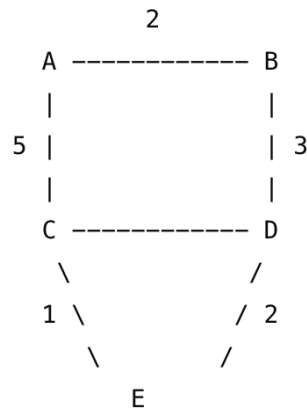


Exercise 2: Link State and Distance Vector Routing

Name: Ashira Chansawang **Student ID:** 6710451496 **Date:** 9/21/2026

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	Infinity, -	Infinity, -
1	B	2, A	5, A	5, B	Infinity, -
2	C	2, A	5, A	5, B	6, C
3	D	2, A	5, A	5, B	6, C
4	E	2, A	5, A	5, B	6, C

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:

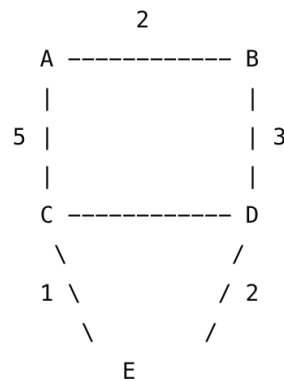
Destination	Shortest Path	Total Cost
B	A-B	2
C	A-C	5
D	A-B-D	5
E	A-C-E	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A–B
C	5	C	A–C
D	5	B	A–B–D
E	6	C	A–C–E

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	$2+0 = 2$	$5+4 = 9$	2	B
C	$2+4 = 6$	$5+0 = 5$	5	C
D	$2+3 = 5$	$5+1 = 6$	5	B
E	$2+5 = 7$	$5+1 = 6$	6	C

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	B	2	A–B
C	C	5	A–C
D	B	5	A–B–D
E	C	6	A–C–E

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

Distance Vector routing.

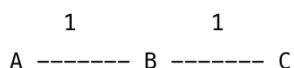
Your answer should describe:

- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

Link State	Distance Vector
C and D detect it first (link goes down on their end)	C and D notice first, mark cost to each other as infinity
They flood a new LSA saying the C–D link is gone	They send updated vectors, but only to direct neighbors
Every router updates its link-state database (LSDB)	Update spreads hop-by-hop, slower than flooding
Each router reruns Dijkstra on the new LSDB	Other routers may still use stale info → temporary wrong/looping routes
New shortest paths found fast, network stays consistent	Converges after enough rounds, can be slow (count-to-infinity risk)

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?

Ans: B loses its direct path to C but still trusts A's old advertised cost (2) except that cost actually depends on going through B itself, so they end up routing through each other.

2. Show how the estimated distance to C can increase over successive iterations.

Ans: DB(C) goes $3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \dots$ climbing by 1 each round as A and B keep re-advertising off each other's last (stale) value.

3. What is count-to-infinity?

Ans: the slow crawl where distance estimates keep inching up instead of the failure being recognized right away.

4. Why does it occur in Distance Vector routing?

Ans: routers only know distance + next hop, not the full path, so they can't tell they're routing back through themselves.

5. Name a mitigation approach. Ans: split horizon with poison reverse

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	Dijkstra's algorithm (SPF)	Distributed Bellman-Ford
Information maintained	Full network topology	Only cost to neighbors
Information exchanged	Link states, flooded to all routers	Full distance table, sent to neighbors only
Algorithm	Each router computes its own shortest paths	Routers update together based on neighbor info
Topology knowledge	Global view	Local view only
Routing updates	Flooded network-wide on change	Sent periodically/on change, to neighbors
Convergence	Fast	Slower
Routing loops	Rare, brief	Common during failures
Count-to-infinity	Doesn't happen	Can happen
Example protocol	OSPF, IS-IS	RIP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

Ans: I'd go with Link State (like OSPF) here. First, it converges way faster since routers flood changes right away and each one just reruns Dijkstra on its own instead of waiting on neighbors like DV does. Second, LS gives every router the full topology, so routing is more accurate and there's no count-to-infinity issue to deal with. Third, OSPF supports hierarchical areas, so with 500 routers you can split the network into areas and keep the flooding/LSDB size manageable instead of one giant flat domain.

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

Ans: LS routers keep the whole topology ($O(N+E)$ info) and flood updates to everyone, so per-change overhead is higher, but each router computes independently with Dijkstra ($O(E + N \log N)$). DV routers only store distance vectors ($O(N)$) and only talk to neighbors, so per-message cost is smaller, but it takes multiple rounds (up to the network diameter) to converge. LS converges much faster and doesn't risk looping forever like DV can. For scalability, LS handles big networks better thanks to hierarchical areas, while DV struggles past a certain size (hop limits, slow convergence). That's basically why large ISPs/enterprises run OSPF/IS-IS instead of RIP for intra-domain routing.